



850 Engineering Hours Back, Every Month:

Context-Aware Agentic Airflow Platform at Wix



Yarden Wolf



HI, I'M

Yarden Wolf

Data & AI Engineer @ Wix

- Developer Platform Group (2 Years)
- Market Intelligence Group (Current)
- Leading AI Initiatives across the Data Engineering Group



[/in/yarden-wolf](https://www.linkedin.com/in/yarden-wolf)

AGENDA

Where we're headed today

01 About Wix & DE's Group

the company & our data scale

02 Local Development

compliant DAGs & debugging, with context

03 Production & AirBot

resolving incidents autonomously

04 Lessons Learned & Next Steps

key takeaways & shift to multi-agent with A2A

THE COMPANY

About Wix

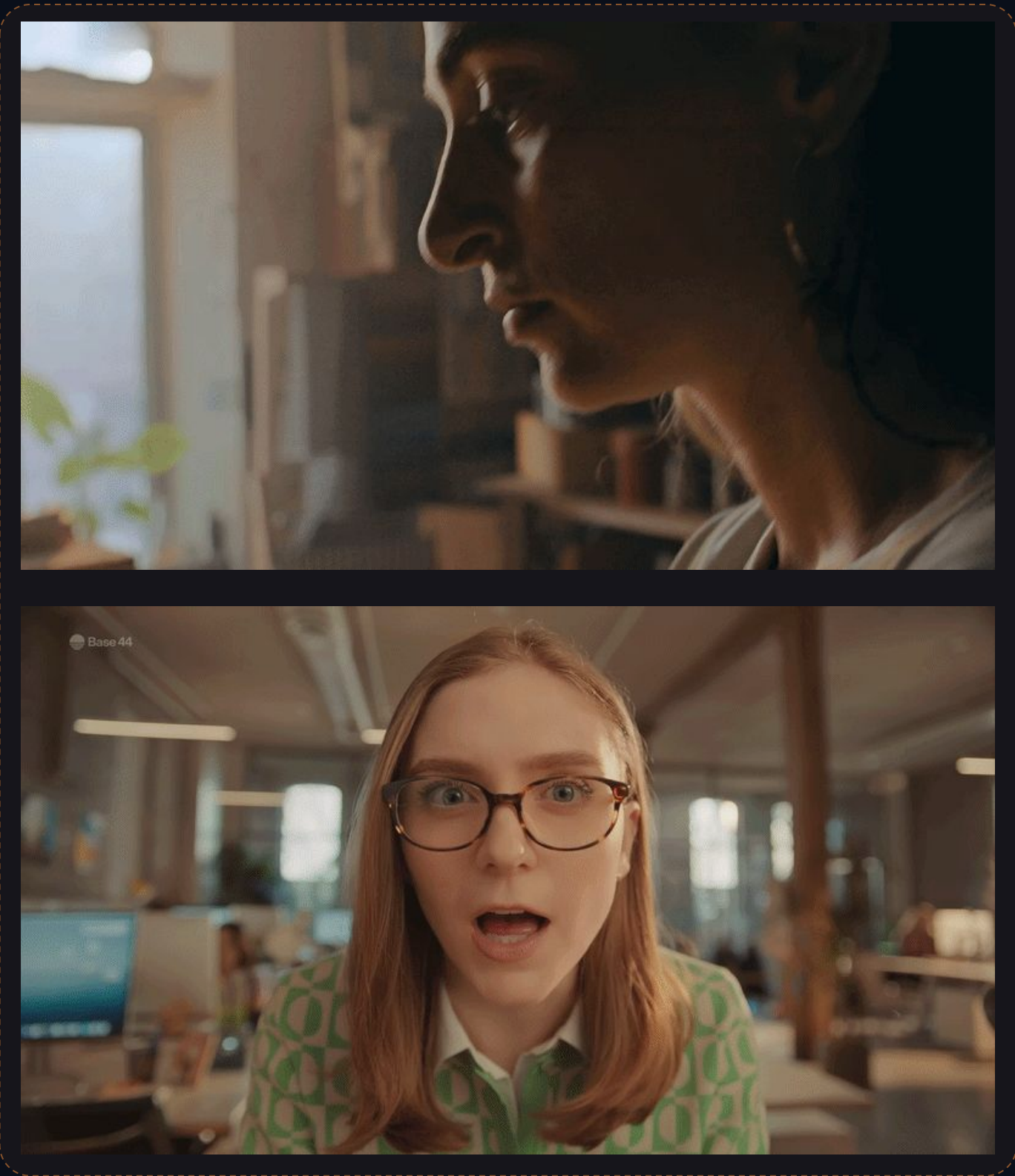
A leading SaaS website builder platform.

~280m registered users globally

~10% of all active cms sites on the web run on Wix

~45% of market share to sites built with simple website builders

~30% **Base44** AI app builders market share in the U.S.



Data @ Wix



People

- 100+ Data Engineers
- 50+ Data Scientists
- 150+ Data Analysts



Data

- 7PiB of Data Lake Storage
- 20TB of Data processed daily
- 2M SQL queries per day



Airflow

- 2 Production Airflow Clusters
- 8,000+ DAGs in production
- 250,000+ Airflow tasks a day

MEET MAYA

Maya vs. The Complexity of Scale



Maya

a composite of every Wix Data Engineer

Maya is good at the job. But three things **hold back** her productivity.

- 01 Ensuring new DAGs satisfy all internal engineering guardrails
- 02 Figuring out why it breaks locally
- 03 Getting alerted at 2 AM when it breaks in production



BATTLE 01

“Just write the DAG.”

How hard can it be at our scale? (100+ Data Engineers)



The context gap

Deploying a DAG requires alignment with proprietary Wix architecture and internal engineering guardrails:

CUSTOM OPERATORS (WIXFLOW)

Using our in-house Airflow operators correctly.

DWH MODELING PATTERNS

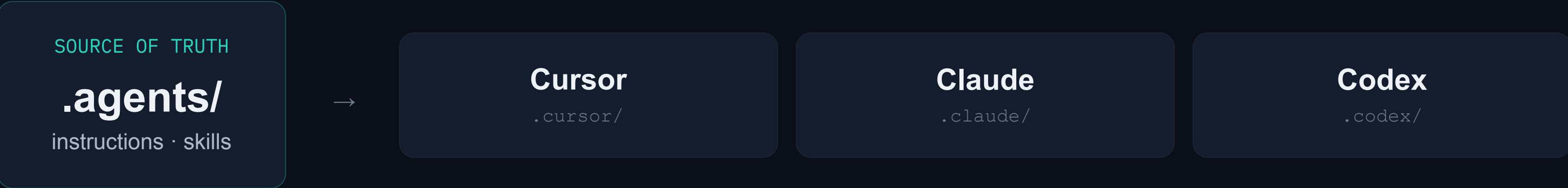
Modeling techniques, Trino & Iceberg SQL expressions, partitioning.

STRICT GOVERNANCE RULES

DWH Table & column naming conventions and governance rules.

One source of truth, for every agent

Authored **skills** as **plugins** at the `.agents` level to project them into whichever framework the engineer chooses.



Getting it to all 100+ engineers: AICM

AI Configuration Manager — our internal **agent agnostic** framework for coding-agent configuration. It **versions** and **distributes** agent context (plugins -> skills & MCP configs) to every repo from a **single source of truth**.

01 One change, distributed to every repo

Without it: updating one skill means a PR in every repo. With AICM: publish once, every repo pulls it.

02 Versioned like a package

Versions locked per repo. A bad context update rolls back in seconds.

03 Engineers aren't the only ones who'll use them

The same team specific skills that guide Maya in her local, **become part of the context for AirBot in production**.

Getting it to all 100+ engineers: AICM

The Step-by-Step **Distribution** Lifecycle

```
wix-astronomer-dags > {} aicm.json > {} workspaces
1  {
2    "targets": [
3      "cursor",
4      "claude-code",
5      "codex"
6    ],
7    "plugins": [
8      "https://github.com/wix-private/coding-agents-plugins/tree/master/plugins/data-eng-pipelines-dev",
9      "https://github.com/wix-private/coding-agents-plugins/tree/master/plugins/data-expert",
10     "https://github.com/wix-private/coding-agents-plugins/tree/master/plugins/data-automations",
11     "https://github.com/wix-private/coding-agents-plugins/tree/master/plugins/etl-kit"
12   ],
13   "skills": [],
14   "lastUpdated": "2026-06-14T00:00:00.000Z",
15   "workspaces": {
16     "./premium": {
17       "plugins": [
18         "data-eng-premium"
19       ]
20     }
21   }
22 }
```



```

  coding-agents-plugins
  coding-agents-plugins
    plugins
      data-eng-pipelines-dev
        skills
          add-gx-quality-checks
          airflow-dag-development
          airflow3-deprecation-fixes
          airflow3-local-testing
          anomaly-detection-onboarding

```



```

  wix-astronomer-dags
  .agents
    skills
      data-eng-pipelines-dev_add-gx-quality-checks
      data-eng-pipelines-dev_airflow3-deprecation-fixes
      data-eng-pipelines-dev_airflow3-local-testing
      data-eng-pipelines-dev_anomaly-detection-onboarding
      data-eng-pipelines-dev_check-duct-cake-ci-status
      data-eng-pipelines-dev_dag-project-structure

```

Battle 01 • Verdict

Guess who wins?



MAYA'S WIN · BATTLE 01

“A DAG that used to take me
a week a year ago now takes
a couple of hours.”

BATTLE 02

“Why does it break locally?”

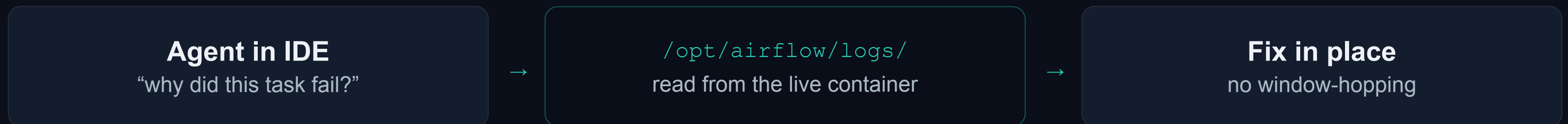
The DAG is written. Now Maya has to debug it.



THE IDEA

Bring the logs to the IDE

Instead of hopping between the Airflow UI and the Terminal — let the agent **read the logs from the running container** and surface them right in her IDE.



THE TOOL

local-airflow-debug

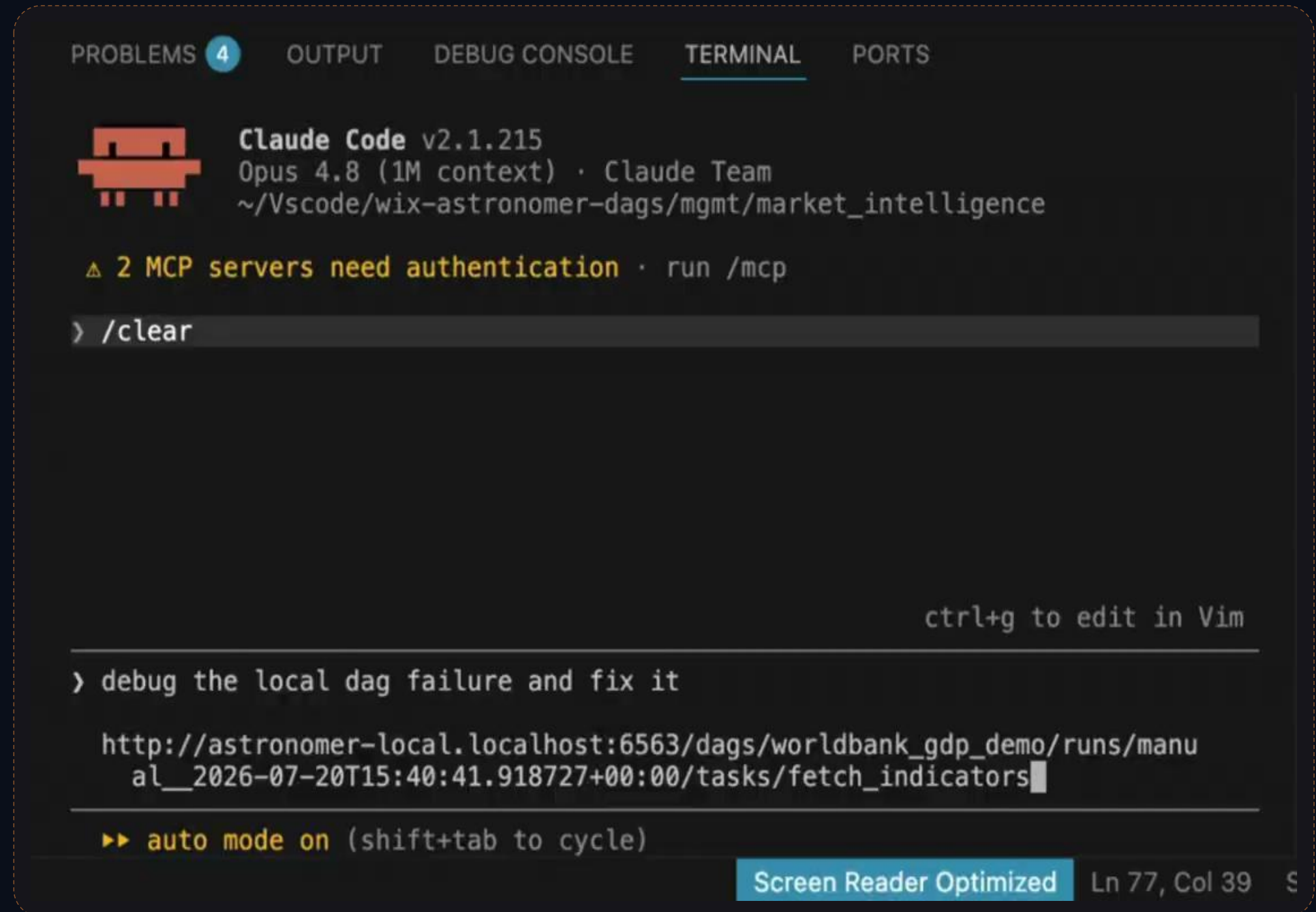
A [skill](#) (via [our skills repo](#)) for the local Astronomer runtime that extracts task logs from Docker containers and utilizes [Astronomer API](#) & [MCP servers](#) to resolve failures.

```
def log_dir(dag: str, run_id: str, task_id: str) -> str:
    return f"/usr/local/airflow/logs/dag_id={dag}/run_id={run_id}/task_id={task_id}"

def log_file(dag: str, run_id: str, task_id: str, attempt: int) -> str:
    return f"{log_dir(dag, run_id, task_id)}/attempt={attempt}.log"

def docker_exec(c: str, cmd: list[str], check: bool = True) -> str:
    return _run(["docker", "exec", c] + cmd, check=check).stdout

def cmd_log(args) -> None:
    env = resolve_env(args.env)
    sched = container(env, "scheduler")
    attempts = list_attempts(env, args.dag, args.run_id, args.task_id)
    path = log_file(args.dag, args.run_id, args.task_id, attempts[-1])
    raw = docker_exec(sched, ["cat", path])
```



PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS

 **Claude Code** v2.1.215
Opus 4.8 (1M context) · Claude Team
~/Vscode/wix-astronomer-dags/mgmt/market_intelligence

⚠ 2 MCP servers need authentication · run /mcp

> /clear

ctrl+g to edit in Vim

> debug the local dag failure and fix it

http://astronomer-local.localhost:6563/dags/worldbank_gdp_demo/runs/manual__2026-07-20T15:40:41.918727+00:00/tasks/fetch_indicators

▶▶ auto mode on (shift+tab to cycle)

Screen Reader Optimized Ln 77, Col 39 S

Battle 02 • Verdict

Guess who wins?



MAYA'S WIN · BATTLE 02

Maya debugs the failing task **without ever leaving the editor** — **no UI, no manual log diving, no guessing.**

Two battles down.

BATTLE 03

The 2 AM Alert

A production DAG just failed. Maya is on-call.



BATTLE 03 • COLD OPEN

**2 AM. A DAG fails. Watch
how Maya handles it.**



THE OLD WAY

5 steps between the alert and the fix

- 01

 **The Alert**

A generic alert fires via Slack.
- 02

 **The Hunt**

Investigate manually through the Airflow UI.
- 03

 **The Deep Dive**

Parse megabytes of logs.
- 04

 **The Synthesis**

Map the error back to a recent change.
- 05

 **The Resolution**

Implement the fix and resolve the incident.

```
@@ -129,17 +129,17 @@ LIMIT 10),
129 129                                AND DATE(CAST('{{ ds }}' AS TIMESTAMP))
130 130                                ) rep
131 131                                JOIN (
132 -                                SELECT CAST(date_created AS date) AS day_date
132 +                                SELECT CAST(created_date AS date) AS day_date
133 133                                , SUM(price_in_usd) AS gpv
134 134                                FROM {{ params.gpv_table }} g
135 -                                WHERE CAST(date_created AS date) BETWEEN DATE(CAST('{{ ds }}' AS TIMESTAMP)) - INTERVAL '30' DAY
135 +                                WHERE CAST(created_date AS date) BETWEEN DATE(CAST('{{ ds }}' AS TIMESTAMP)) - INTERVAL '30' DAY
136 136                                AND DATE(CAST('{{ ds }}' AS TIMESTAMP))
```

The old way of handling Airflow failures was **Slow**, **Repetitive**, and **Tedious**.

THE NEW WAY

Meet AirBot



Our Slack-native AI agent that turns an alert from a notification into an **automated fix**.

From alert to suggested fix — automatically



SPECIALIZED, NOT GENERIC

AirBot knows our operators

A **keyword registry** automatically routes errors to specialized **handlers** based on the **operator** and **error type** of the failure.

DDS (Data Dependency Sensor)

Walks lineage to the root-cause table and pages each owning team in its own channel.

TRINO

SQL failures and query-performance optimization.

SPARK

Resource analysis and big-data performance tuning.

PYTHON

Script error diagnostics across pod and venv operators.

DATA QUALITY (GX)

30+ data-quality tests → auto diagnostic SQL via Trino.

WIXFLOW (IN-HOUSE)

Our internal custom operators & sensors family.

HOW IT FIXES A CODE ERROR

It fixes the code like a developer in our repo

Orchestrated by [LangChain](#) and powered by [Claude Agent SDK](#) for code failures — AirBot acts as [Claude Code CLI](#), operating inside a [live repo checkout](#), leveraging the same [team skills repo](#) that Maya uses locally.

STEP 1 · ANALYZE · READ-ONLY

Find the root cause

Clones the repo, search, read and load the repo's own `CLAUDE.md` + team folder skills — so fixes match our conventions.

human in
the loop
approves



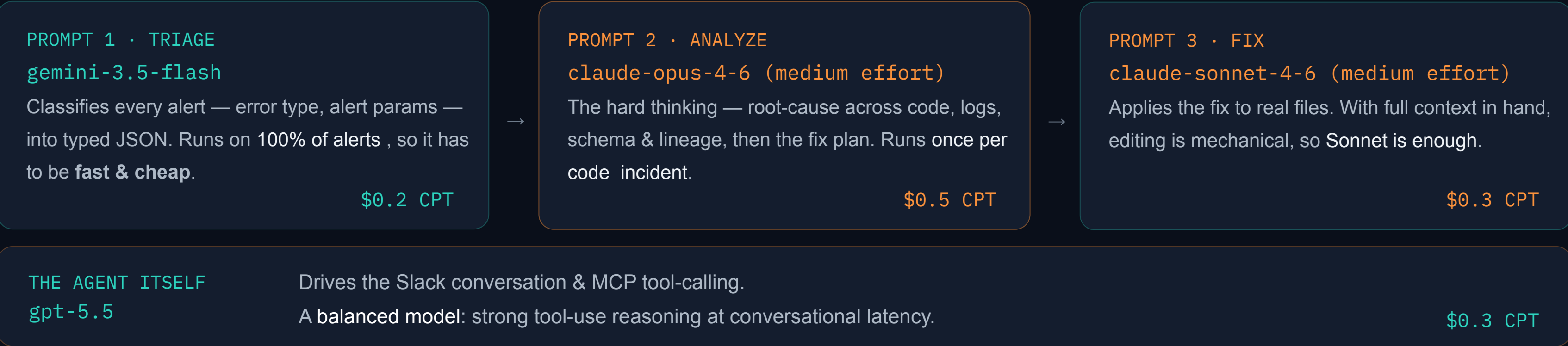
STEP 2 · FIX · WRITE

Open a PR

Edits the files in place; the resulting `git diff` becomes a PR.

The right AI model for each step

Each alert runs multiple LLM calls. We pay premium reasoning rates **only when necessary**.



HOW IT ACCESSES

A secure set of actions, bound to a single service account

Slack Socket Mode

WebSocket integrated with our internal serverless framework

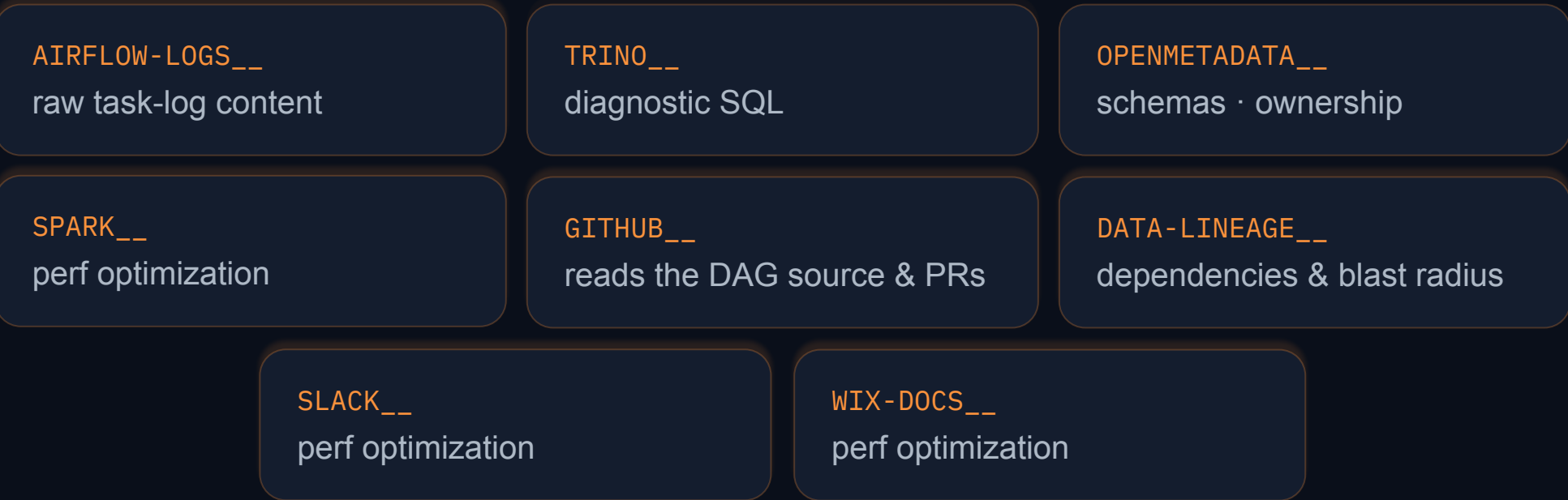
MCP service account

Gates all MCP servers through our internal MCP gateway.

wix-airflow-client

Access the task logs **direct from S3** (IRSA auth) in one call.

8 MCP SERVERS CONNECTED TO AIRBOT



IT LEARNS

Each incident enriches the context for the next.

Every resolved failure is embedded into **Wix's retrieval-service KB** (Vespa DB) with the user feedback.

When a similar error appears, AirBot recalls the closest past incidents ($\geq 70\%$ match) and folds them into its analysis.

[illegible]

Spark Resources

Thread



DE Slack Bot

APP

Friday at 3:53 AM

Failed Dag: pbw_wp_transactions_etl_delta, Task: load_data_delta.

```
AirflowException('
===== ERROR ROOT CAUSE =====: Executor containers are killed with exit code 137 due to out-of-memory (OOM) errors during execution, causing job abort after multiple retries.
===== ERROR TYPE =====: memory (OOM) error
===== SOLUTION =====: Increase executor memory allocation and/or optimize data processing to reduce memory usage; consider tuning Spark memory configs and repartitioning data to avoid large shuffles.')
```

Full log in https://cmbjlwjf30d9001pid9zp8qxc.astronomer.run/debvx117/dags/pbw_wp_transactions_etl_delta/runs/scheduled_2026-02T17:02:00+00:00/tasks/load_data_delta?try_number=4



cmbjlwjf30d9001pid9zp8qxc.astronomer.run



View the Airflow UI on Astro



DAG



pbw_wp_transactions_etl_delta

6 replies



AirBot

APP

Friday at 3:53 AM



Hey there! I will take a look at this error. Please wait...

I found the root cause of this failure

Root Cause: `Executor containers are killed with exit code 137 due to out-of-memory (OOM) errors during task execution, causing job abort after multiple retries.`

Operator Type: `Spark`

Error Type: `Memory`

Merged

heorhii merged 1 commit into master from airflow_alerts_manager_pbw_wp_transactions_etl_delta_2026_...

2 days ago

Conversation 1

Commits 1

Checks 10

Files changed 1

All commits

0 / 1 viewed

Submit comments

Filter files...

...eplications/configs/wp_transactions.py

+6



Yarden Wolf

Wednesday at 1:59 PM

[@Heorhii \(Gosha\) Voskoboinyk WDYT?](https://github.com/wix-private/wix-astronomer-dags/pull/11191/changes)



Heorhii Voskoboinyk

Wednesday at 2:36 PM

Hey, looks good



1



Connection Failure → Auto Clear



Rootly

AGENT

Today at 11:02 AM

New alert from Api (edited)



Resolved by DE API Key v3 from API | July 12 at 11:32 AM IDT



P1 |  Not noise |  See alert in Api

Failed Dag: wix_code_monitoring_dashboard, Task: logs_tg.create_logs_delta.

```
AirflowFailException('Trino query #20260712_080051_24029_deeda failed:
java.io.IOException: Failed to open S3 file: s3a://wix-bi-iceberg-warehouse/wix_code.db/runtime_telemetry_log/data/last_modified_bucket=177698880000/000000_00/0000_
Show more
```

Services

DE DP - P1 critical alerts

Teams

DE Developer platform

Responders

DE API Key v3



Escalate



Mark as Noise

Show more

Added by Rootly

5 replies



AirBot

APP

Today at 11:15 AM



Hey there! I will take a look at this error. Please wait...

I found the root cause of this failure

Root Cause: `Trino query failed due to an infrastructure connection error while trying to read the Iceberg split file. The operation failed to open the file: 's3a://wix-bi-iceberg-warehouse/wix_code.db/runtime_telemetry_log/data/last_modified_bucket=1776988800000/000000-5174773-57f41690-f3c7-4f51-baeb-fb141d26280a-0-00001.parquet' for target table 'prod.monitoring.logs_log_delta' reading from`

Task Tries

1 

2 

All Log Levels

All Sources

Log Settings





rs: None

ERROR

Task failed with exception

```
▼AirflowFailException: Trino query #20260712_080051_24029_deeda failed: java.io.IOException: Fail
File "/usr/local/lib/python3.11/site-packages/airflow/sdk/execution_time/task_runner.py", line 11
File "/usr/local/lib/python3.11/site-packages/airflow/sdk/execution_time/task_runner.py", line 15
File "/usr/local/lib/python3.11/site-packages/airflow/sdk/bases/operator.py", line 417 in wrapper
File "/usr/local/lib/python3.11/site-packages/wixflow/operators/trino.py", line 466 in execute
▼TrinoQueryError: TrinoQueryError(type=INTERNAL_ERROR, name=GENERIC_INTERNAL_ERROR, message="java
File "/usr/local/lib/python3.11/site-packages/wixflow/operators/trino.py", line 455 in execute
```

DDS Alerting

Thread



DE Slack Bot

APP

Jun 15th at 9:09 AM

Failed Dag: gross_model_prediction, Task: dds_dependencies.

```
AirFlowSensorTimeout('Sensor has timed out; run duration of 22152.626795 seconds exceeds the specified timeout of 21600.0.')
```

Full log in https://cmbjlwjf30d9001pid9zp8qxc.astronomer.run/debvx117/dags/gross_model_prediction/runs/scheduled_2026-06-14T00:00:00+00:00/tasks/dds_dependencies?try_number=1



cmbjlwjf30d9001pid9zp8qxc.astronomer.run



View the Airflow UI on Astro



DAG



gross_model_prediction

4 replies



AirBot

APP

Jun 15th at 9:10 AM



Hey there! I will take a look at this error. Please wait...

I found the root cause of this failure

Root Cause: `The DDS sensor for dds_dependencies` timed out after 22152.626795 seconds, exceeding the configured timeout of 21600.0 seconds. This indicates that one or more dependency tables did not update within the expected timeframe. Specifically, the DDS API response time for 'prod.mgmt_secure.earnings_collections_transactions' (2026-06-14 10:57:22.211000+00:00) was not later than the Adjusted Data Interval End (2026-06-15 00:00:00+00:00).`

Operator Type: `Dds`

Error Type: `TableNotUpdated`

I've started checking the error. Please wait for me to respond with a solution.

 New notifications sent to: [#stores-data-quality \(view alert\)](#), [#ps-data-engineers-alerts \(view alert\)](#)

Root Cause Tables: `prod.ecom_temp.gpv_by_line_item` (owner: ecom-platform-data, last updated: 2026-06-14 06:44:49.928000, [creation job](#)), `prod.premium_secure.financial_transactions_channels` (owner: premium-data, last updated: 2026-06-14 07:10:49.909000, [creation job](#))

Affected Table(s): `prod.mgmt_secure.earnings_collections_transactions` (owner: mgmt-strategic-finance-data, last updated:



AirBot

APP

Jun 15th at 9:10 AM

Dear Owner,

You are the owner of the following root cause table(s) for the DDS failure task in the DAG [gross_model_prediction](#):

- Root cause table(s): `prod.premium_secure.financial_transactions_channels` (last updated: 2026-06-14 07:10:49.909000) ([creation job](#))

- Affected table(s): `prod.mgmt_secure.earnings_collections_transactions` (last updated: 2026-06-14 10:57:22.211000)

Please make sure to update the above table(s) as soon as possible.

 [View original alert thread](#)

2 replies



Boris Slutski

Jun 15th at 9:33 AM



@ldan Rachmanov - I have a proposal to improve the performance of the Quix job.

Ping me when you're available.

About Wix & DE's Guild

Local Development

Production & AirBot

Next Steps

WIX

THE NUMBERS

What AirBot bought us

~850

engineering hours saved per month

≈ 5 full-time data engineers = \$50k!

70

Slack Channels (25 teams)

~\$1

per full automated workflow

15m+

MTTR - per incident

73 / 33 (45%)

PRs proposed / merged last week

Cost by model - last week				
MODEL	REQUESTS	COST	AVG / THREAD	% OF COST
claude-opus-4-6	1,787	\$145.59	\$0.0494	54.7%
claude-sonnet-4-6	1,084	\$49.22	\$0.0167	18.5%
gemini-3.5-flash	3,864	\$38.34	\$0.0130	14.4%
gpt-5.5-2026-04-23	397	\$33.01	\$0.0112	12.4%

MAYA'S WEEK, REWRITTEN

Closing the loop



ADOPTION

Onboarding a new team takes two steps

1

Add AirBot to your Slack alerts channel

That's the whole integration. Every alert that lands there is now triaged automatically.

2

Create a team plugin in AICM

Drop it in the team's folder — your conventions, operators, and known failure patterns.

That's it. From now on, **AirBot handles the team's alerts** — with a specific team's context behind every suggested fix.

Lessons learned

01 Context is everything

Garbage in, garbage out. Agent quality is capped by the context you feed it.

02 Model cost is a design constraint

As models get more powerful, they get more expensive. Route cheap models to triage, reserve deep reasoning for hard failures.

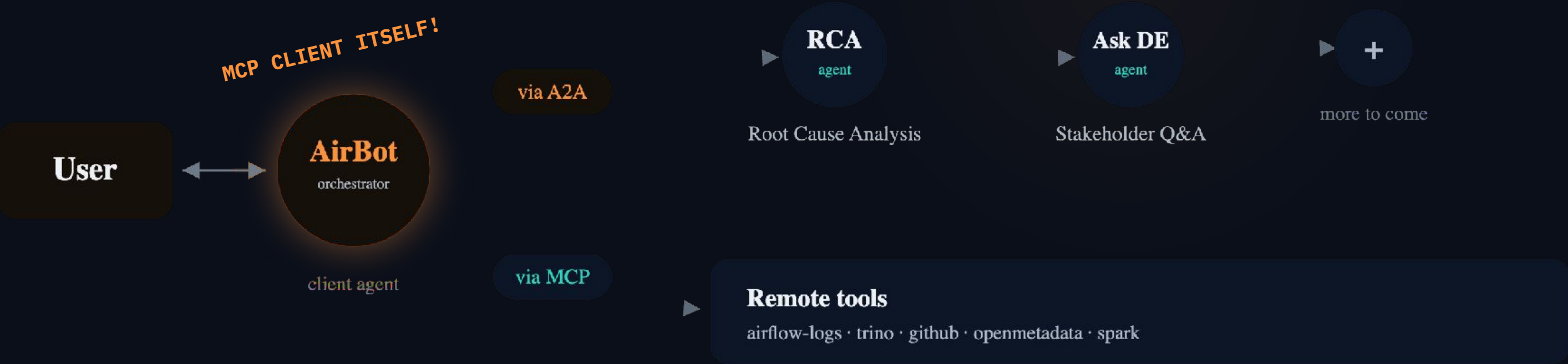
03 Human-in-the-Loop

Show the **reasoning process**, incorporate **user feedback**, and keep a **human in the loop**; autonomy is granted by users only after the agent proves itself.

WHAT'S NEXT

From one agent to A2A approach

We've started moving to an **A2A (agent-to-agent)** architecture — AirBot as the main brain and **orchestrator** of a standalone agents.



PROS:

- CROSS-FRAMEWORK**
LangGraph, Claude SDK, ADK — one open standard.
- ORG DECOUPLING**
Teams ship agents independently
- NETWORK-NATIVE**
HTTP Async protocol.

Questions?



LET'S CONNECT

Yarden Wolf

Data & AI Engineer @ Wix.com